SYSTEMS REQUIREMENT DOCUMENT

DPP Data Pipeline

GROUP **RNN**

PASCUAL, NICANOR FROILAN
RAMOS, NATHAN RAFAEL
TORRES, ALOYSIUS REVE

To be served to:
PHILIPPINE MEDIA MONITORING LABORATORY (PMML)

PROJECT BLOG:
https://cs191-2627a-rnn.github.io

CS 191 WFR-1
Prof. LIGAYA LEAH FIGUEROA

Web App Requirements Model

I. Content Model

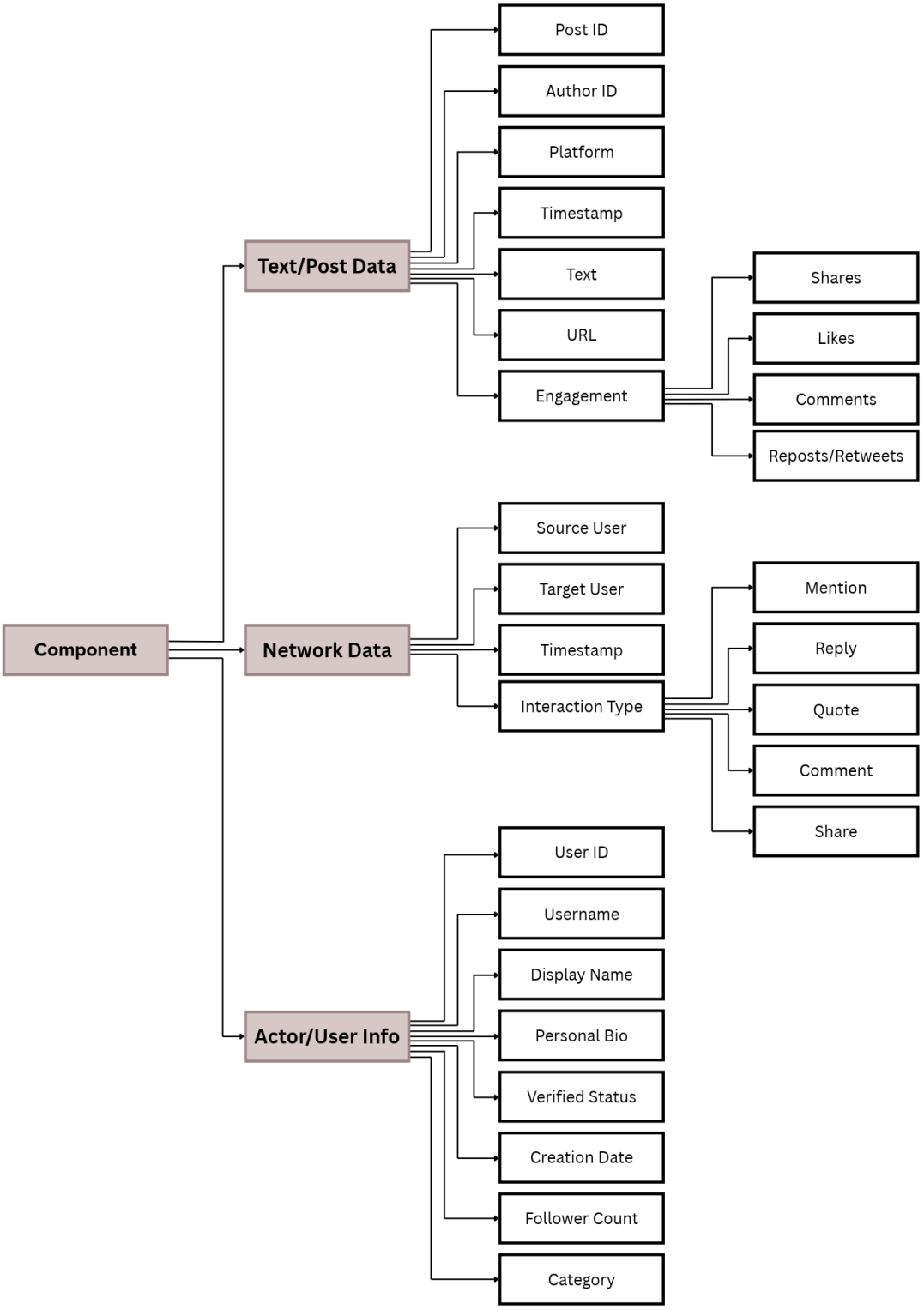


Figure 1: Data Tree of Content Objects for the DPP Data Pipeline (Grey - Content Objects, White - Data Items)

II. Interaction Model

The DPP Data Pipeline has three (3) actors:

- **Researchers** (Initiating): The product user. They require the pipeline to streamline the processing and analysis of data the DPP project needs.
- **System Administrators** (Initiating): These are users who need to have the ability to manage (turn on, shut down, monitor, rollback, etc.) the pipeline manually if needed.
- **Platform APIs** (Participating): Actors from which data will be collected from.

More specifically, the final product has the following use cases based on what each actor requires / contributes:

Researchers	
Use Case	Description
Input Data Into Pipeline	The pipeline can take in data from platform APIs the researchers pass onto it.
Parse and Process Datasets	Automate the raw text cleaning and processing through the pipeline.
Automate Data Validation	The pipeline should be able to check if there are missing key fields in the exported data.
Source and Temporal Analysis	Analyze the trends and engagement of posts through different time windows.
Network Analysis	Analyze through the use of network graphs the connections between data.
Message Analysis	Analyze the data to determine which keywords/phrases are most visible.
Visualize Analysis Through Graphs	Project the analyzed data through plotted graphs, mapping relevant relationships and networks.
View Analysis Through Dashboard	The user should be able to see the data through an interactive dashboard interface for exploration.
System Administrators	
Use Case	Description
Manual Configuration	The administrator can make changes to the system settings depending on need.
System Monitoring	A view where the administrator can watch over the system and any errors that may occur.

Platform APIs	
Use Case	Description
Retrieve Social Media Data	The system requests data from Apify or through the platform APIs themselves.
Return Extracted Data	The service returns the requested data which can be utilized for pre-processing by the system.

Table 1:Use Cases from each Actor

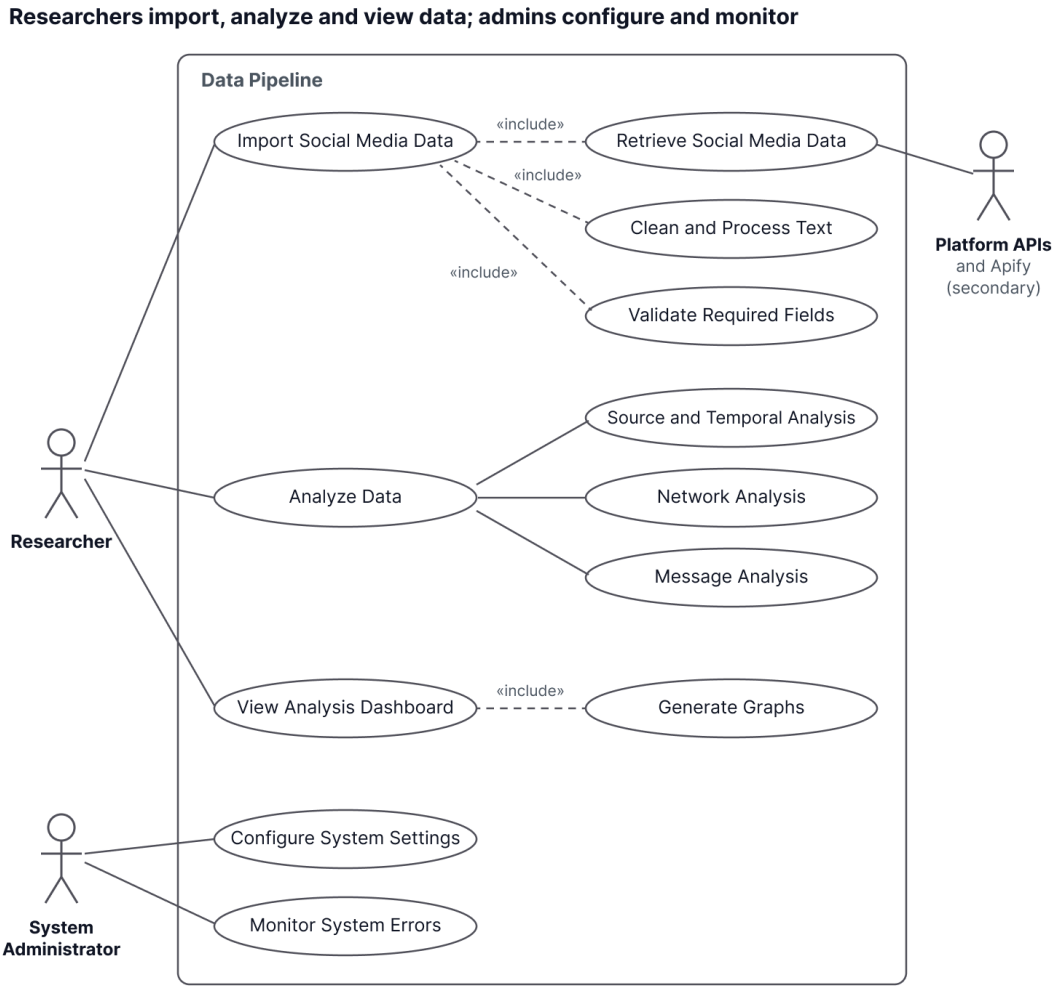


Figure 2: Use-case diagram for the DPP Data Pipeline

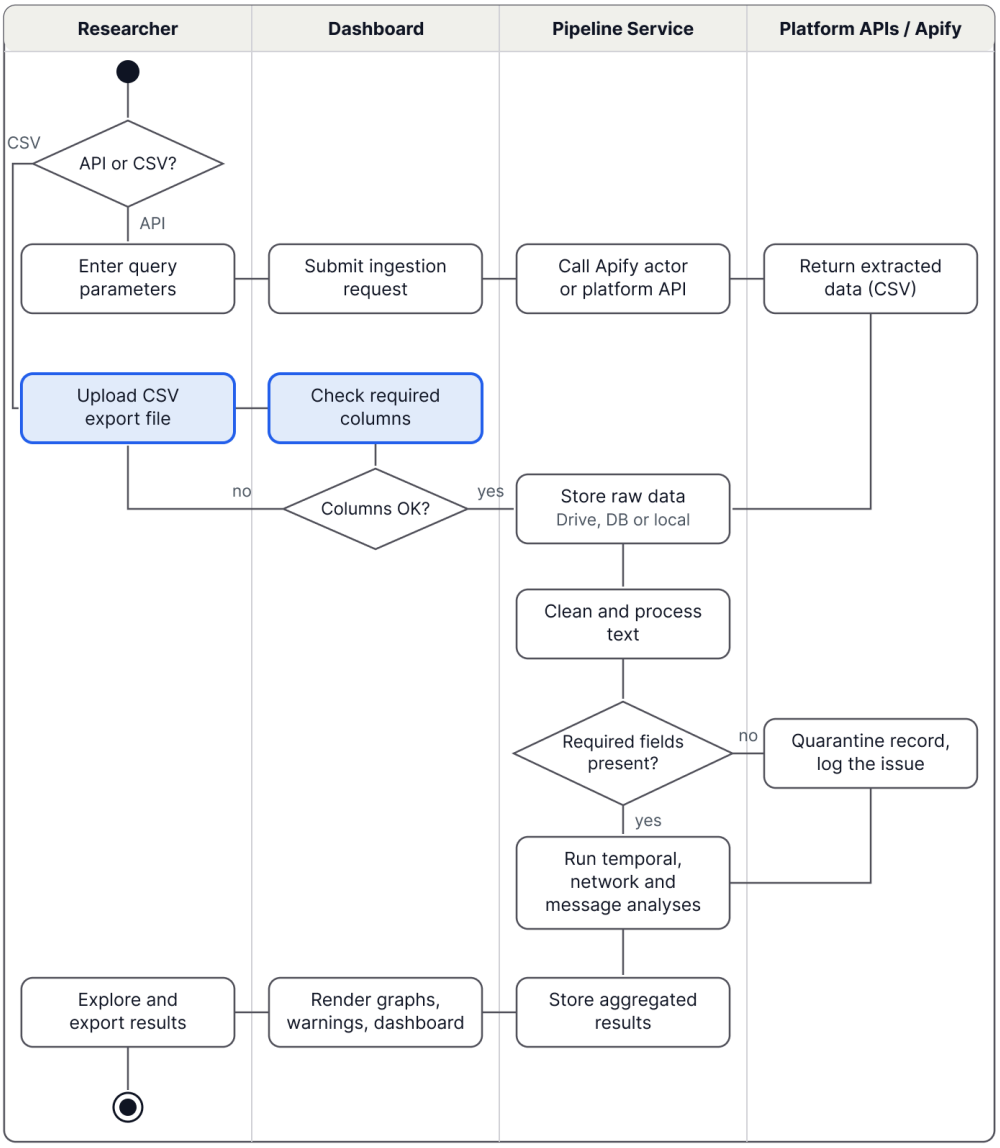


Figure 3: Swimlane Diagram

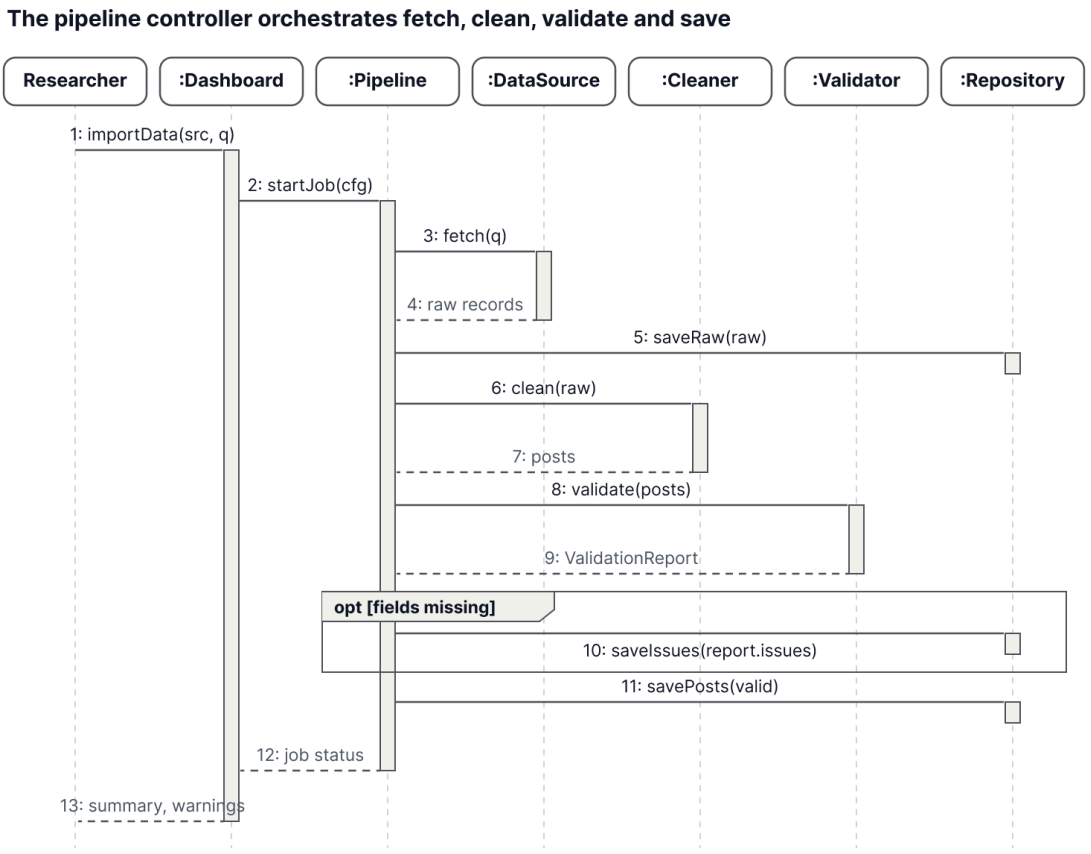


Figure 4: Sequence Diagram

III. Configuration Model

Subsystems and Allocations		
Computing Environment	Modules / Components	Responsibilities
User Machine	User Interface	Upload datasets, configure analysis parameters, and view interactive visualizations.
Web App Server	App Backend, App Frontend, API	Handle user requests, validate uploads, coordinate processing workflows, and return results to the client.
Processing Server	Data Processor, Data Analyzer, Visualization Generator	Parse, clean, and normalize datasets, perform analysis, and construct visuals.
Database / File Storage	Data Storage, Results Storage	Store uploaded datasets, cleaned records, and analyzed outputs.

External Services	Apify Actors, Supported Platform APIs	Supply extracted social media data when required.

Table 2: Possible Subsystems and their associated Modules and Responsibilities

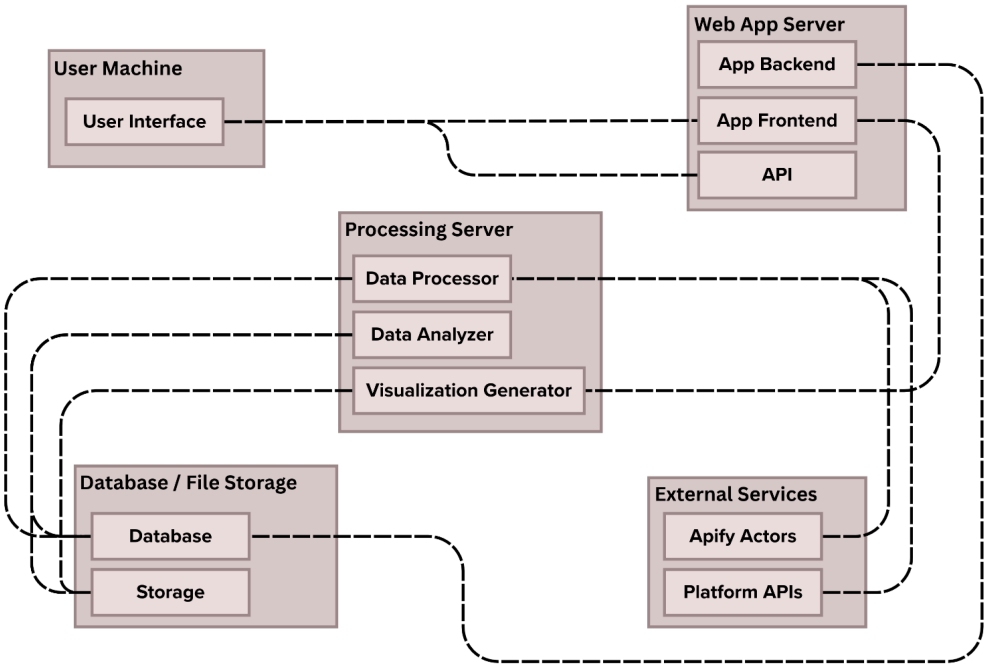


Figure 5: UML Deployment Diagram

Web App Architecture and Design

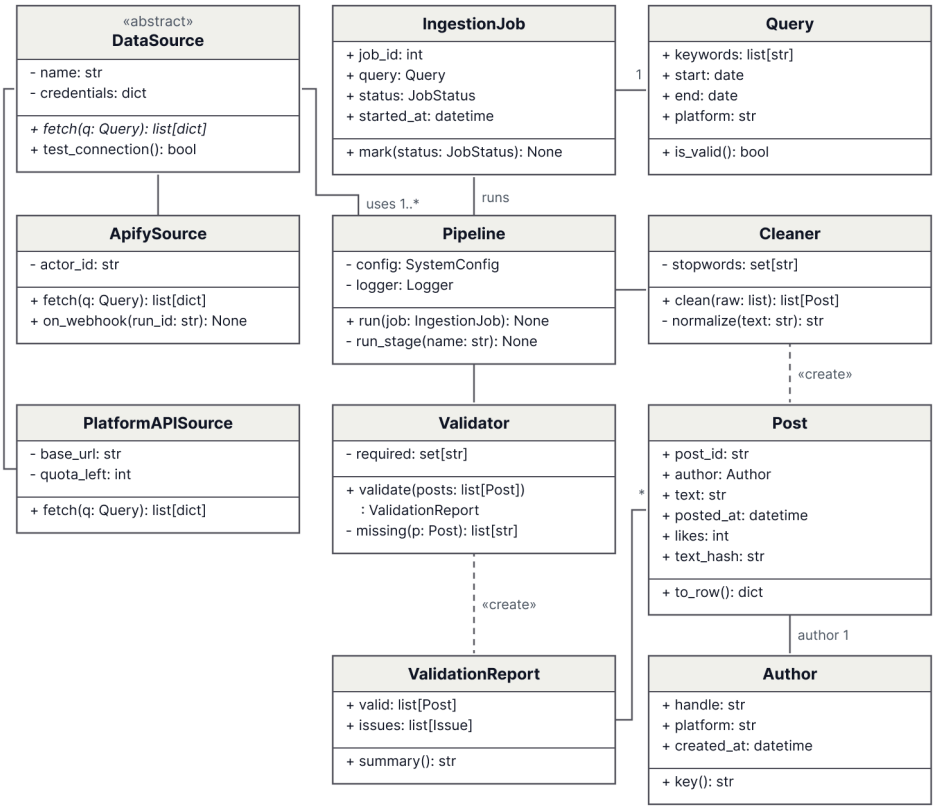
I. Architecture Design

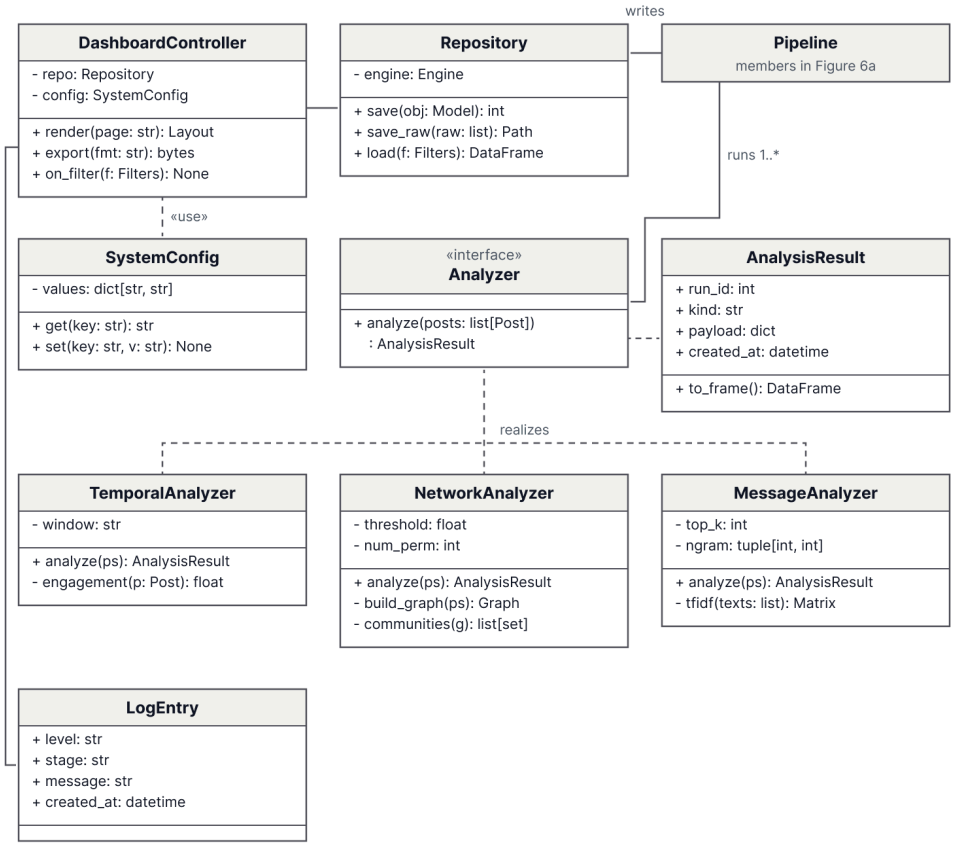
Describe the architectural styles used in your web app (Section 13.7). Show how your design addresses the technical attributes of the "quality requirement tree" (see Figure 13.1). Document the architectural decisions made for your project. For more details, see sample Architectural Decisions artefact and chapter 9 of our textbook.

II. Content (Data) Design

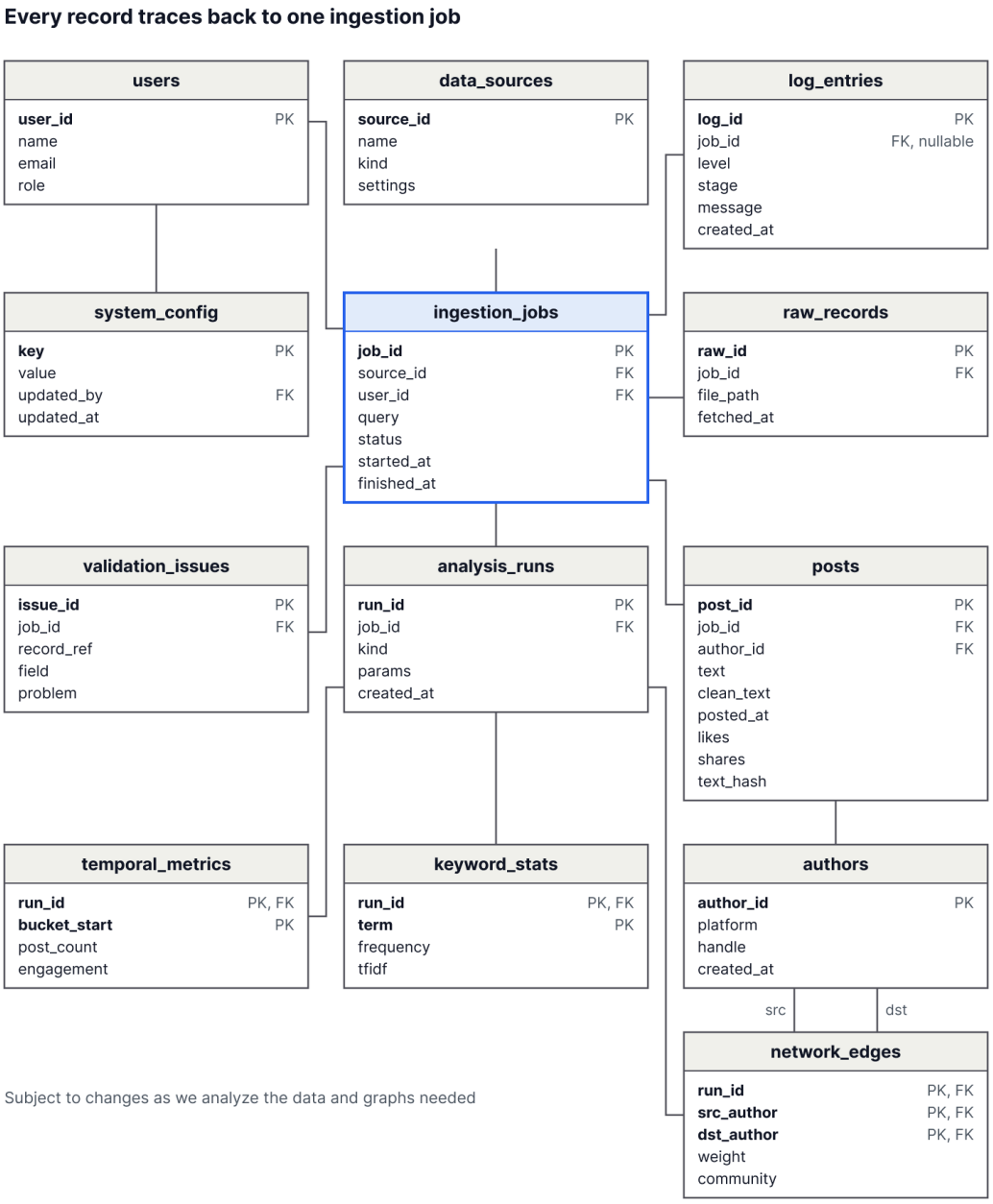
- **Class Diagrams**

The Pipeline class coordinates sources, cleaning and validation



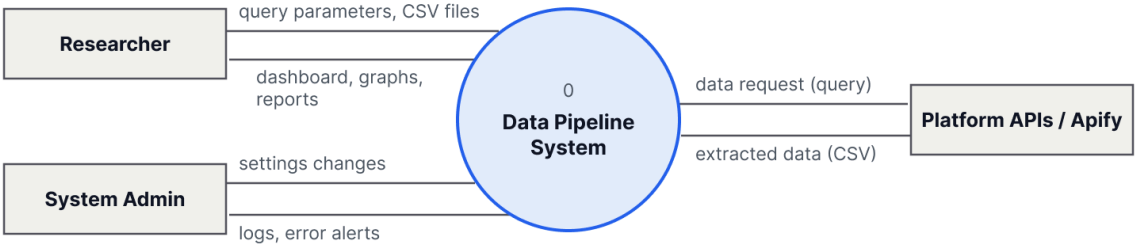


- **Entity-Relationship Diagrams**

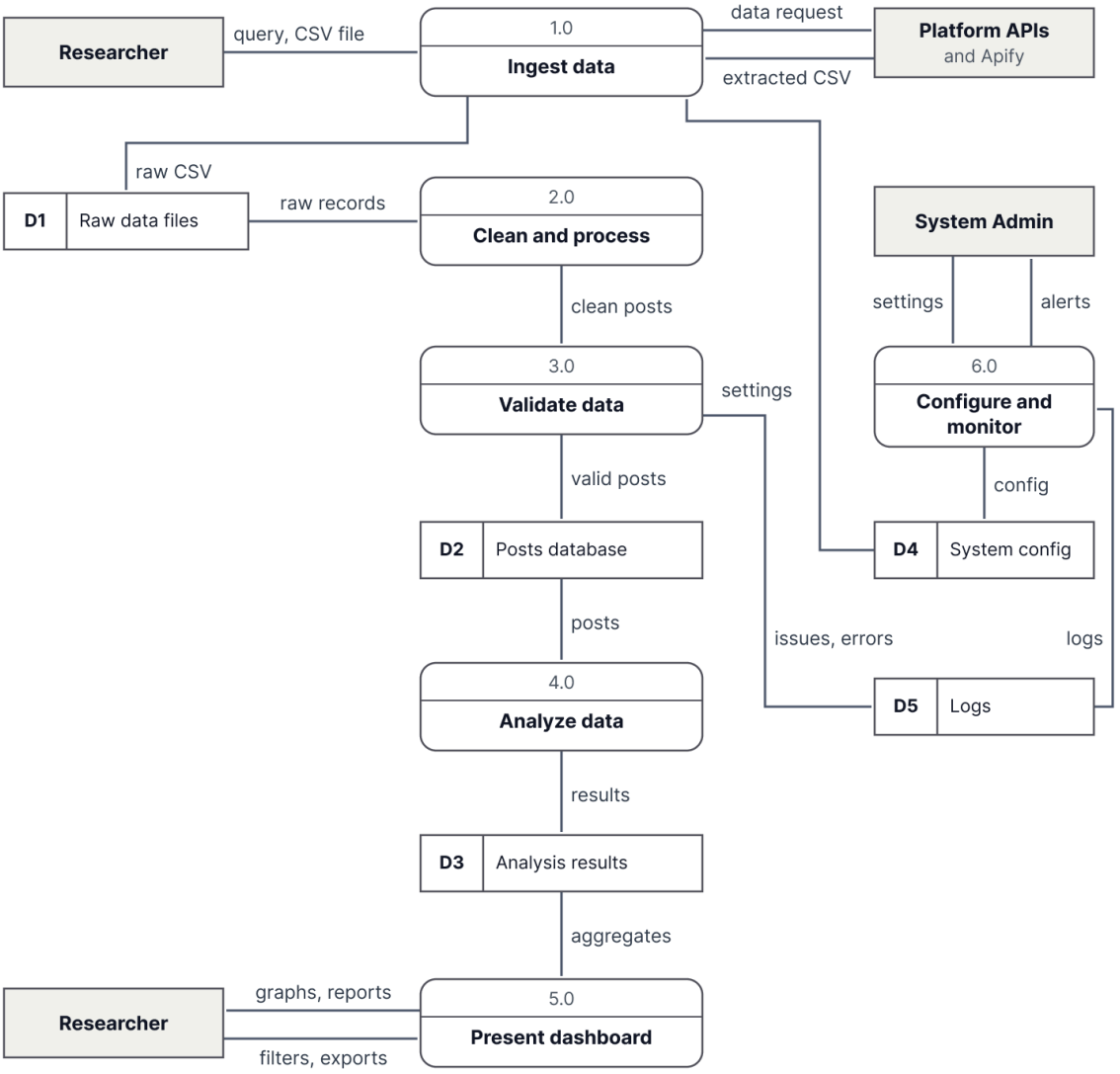


- **Data Flow Diagrams**

Level 0: three external entities exchange data with one system

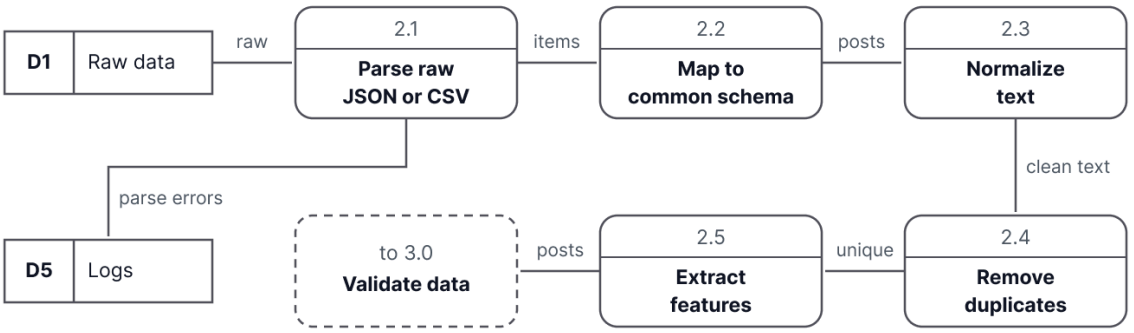


Level 1: data moves through six processes and five stores



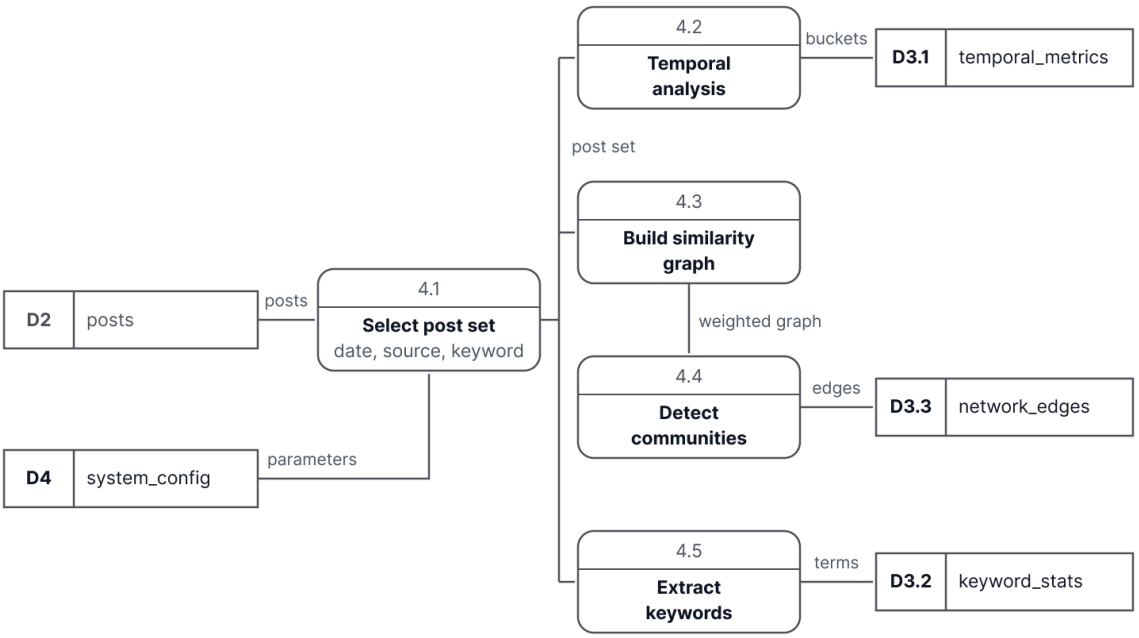
Processes 1.0 to 5.0 also write log events to D5; those flows are omitted for clarity.

Level 2 of 2.0: five steps turn raw JSON into unique, normalized posts



Features: hashtags, mentions, outbound domains, text_hash. Dashed box is outside this level.

Level 2 of 2.0: one post set feeds three independent analyses



D3.1 to D3.3 together form D3 Analysis results in Level 1, which 5.0 reads.

III. Component-level Design

Specify the system components – define the data structures of all local data objects and algorithmic/procedural detail of all processing within a component (use the sub-questions below as a guide). Include interfaces (see Figure 8.5) and dependencies (e.g., sharing, flow, constrained - see Section 9.5.2). Will you be using any COTS components? (See chapter 10). Refer to Chapter 8 and Chapter 10 as you discuss design concepts that you have implemented.

- **Algorithms**
  Describe complex functional/processing details of the web app through a UML activity diagram.

For example, when computing a motion trajectory for an animate figure in a game, you may use some numerical or computer-graphics algorithms. Will the path coordinates be pre-computed and stored in a look-up table or will they be computed using a spline interpolation algorithm? Or, when assessing stock market movements, you may be using statistical algorithms.

- **Data Structures**
  Does your system use any complex data structures, such as arrays, linked lists, hash tables, or trees? Specify your criteria for deciding what data structure to use, e.g., performance vs. flexibility.

IV. Interface Elements / Usability Design

The following set of graphics (Figure A to G) below will be the template for how the interface may look like for the project. Each page has the navigation links at the top since the content should be the priority. To indicate which interface the user is in, the corresponding navigation page title will appear on the header, and will disappear in the navigation bar.

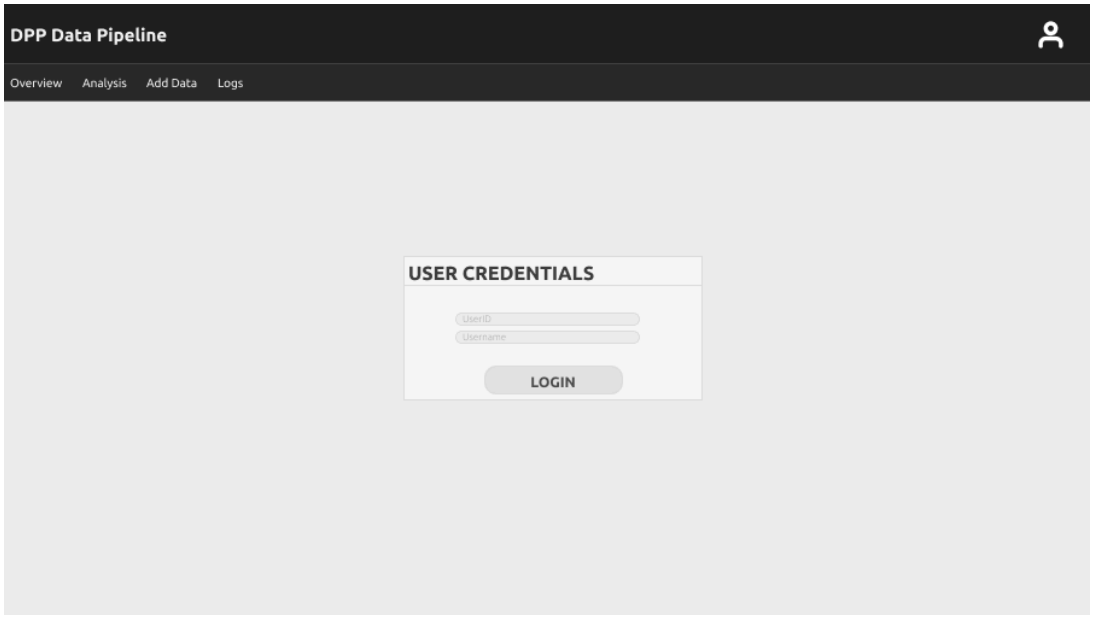


Figure A: Login Interface

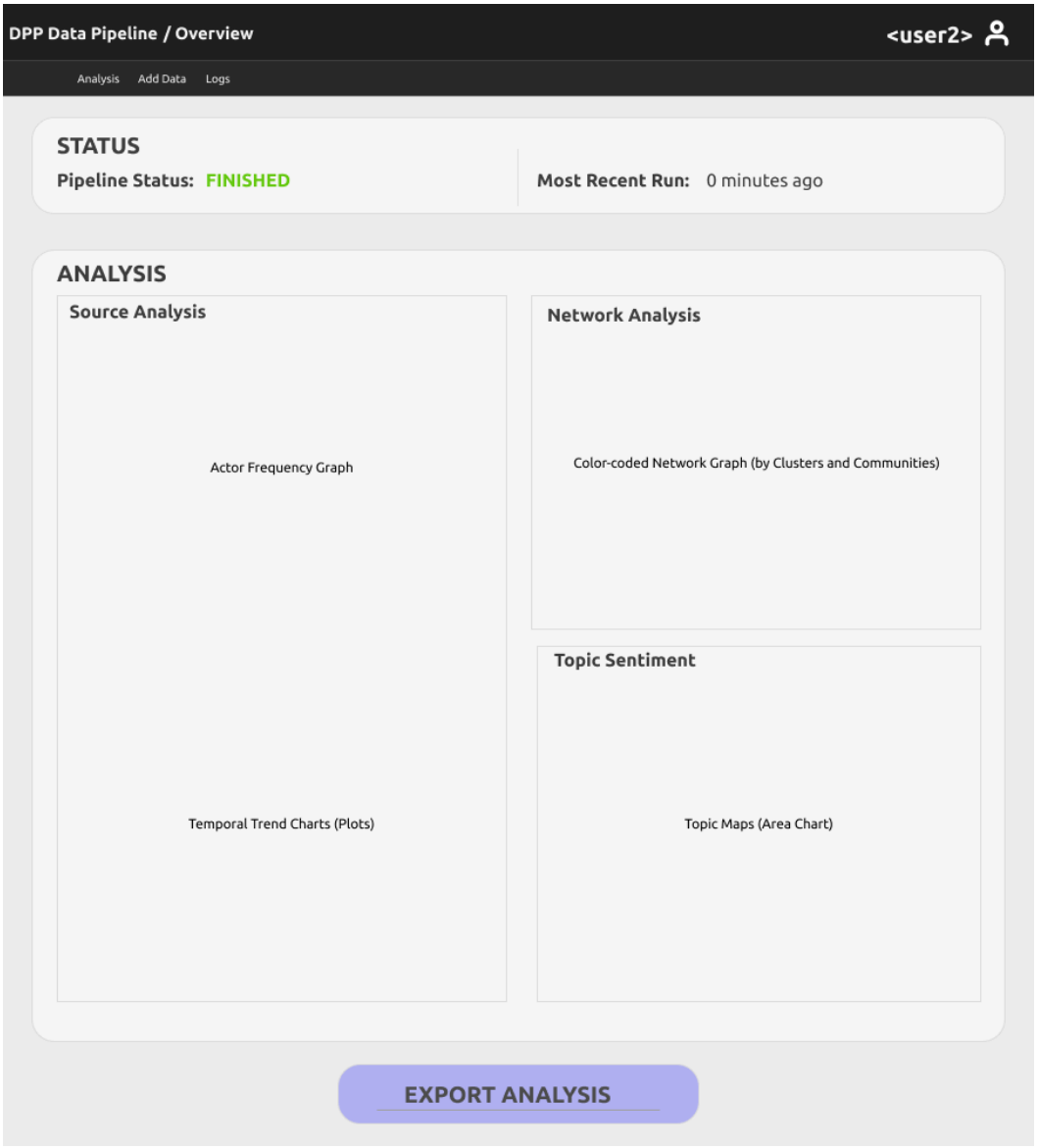


Figure B: Overview Interface



Figure C: More Detailed Analysis Interface

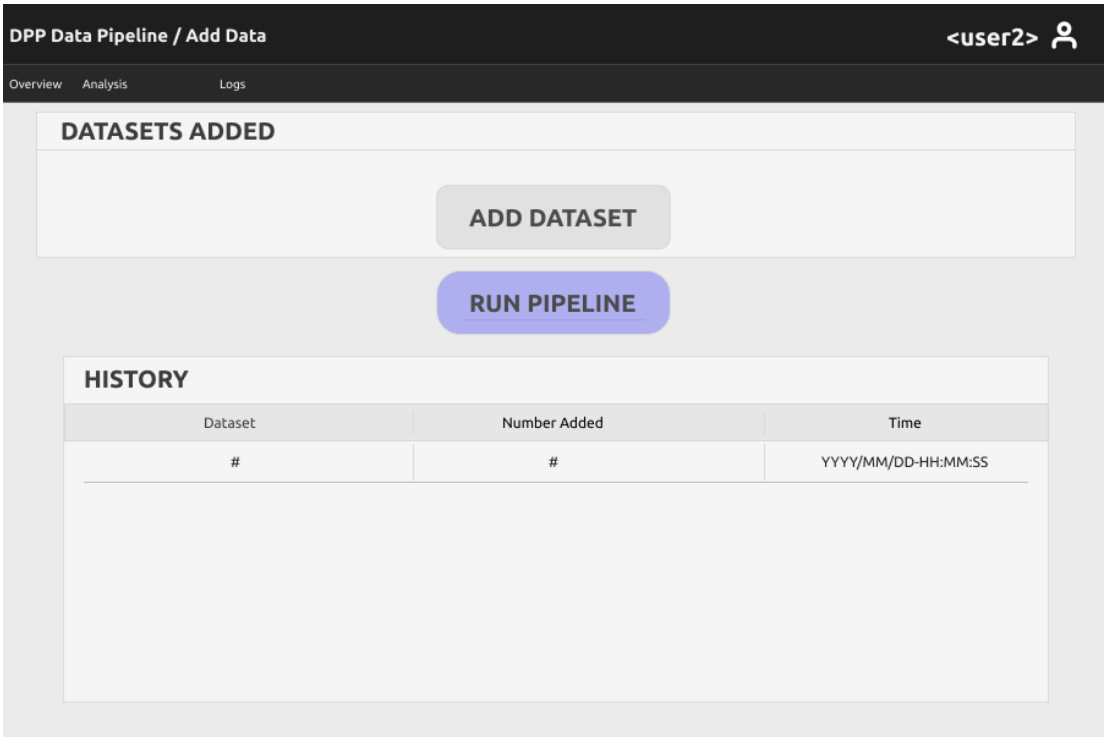


Figure D: Add Dataset Interface

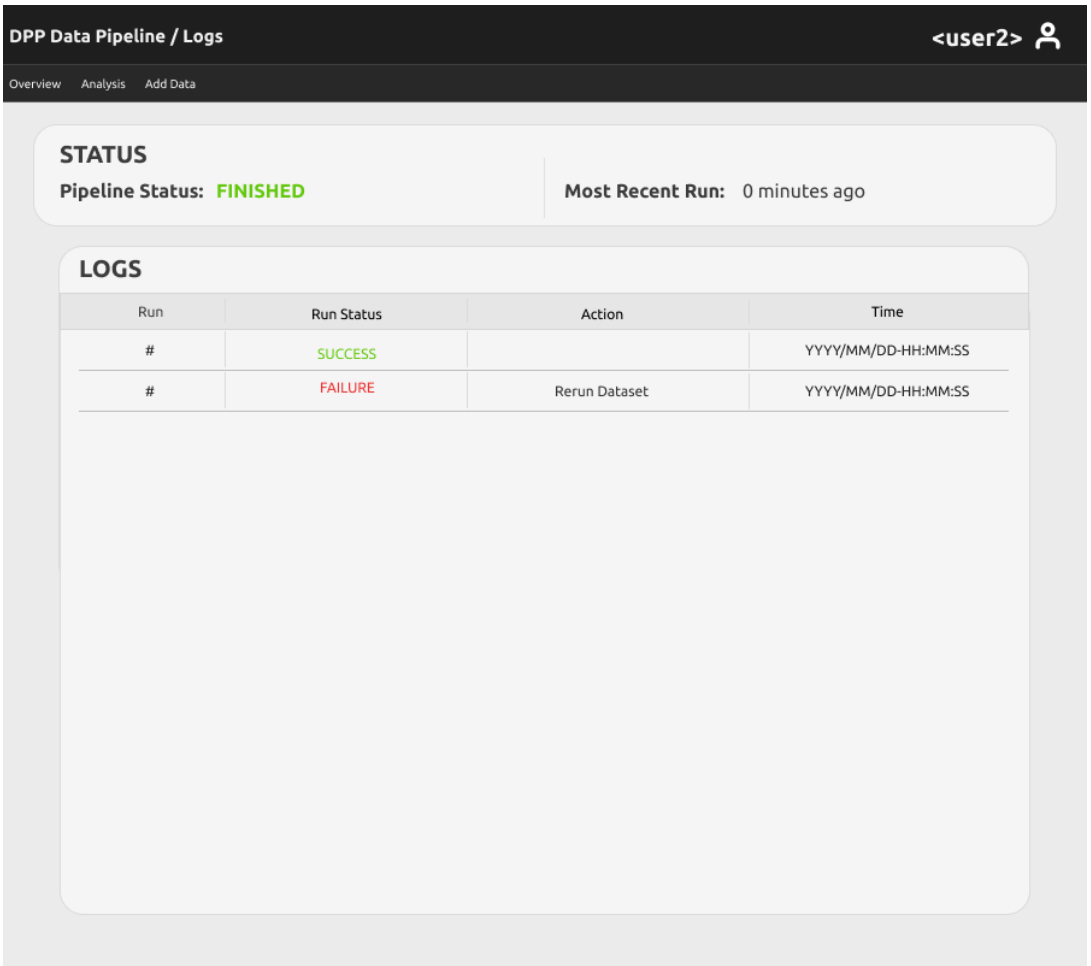


Figure E: Pipeline Logs Interface

In addition, the system administrator, who is a privileged user, has access to 2 more navigation pages dealing with system control. The administrator page can be distinguished by its dark blue header contrasting the black header for normal users:

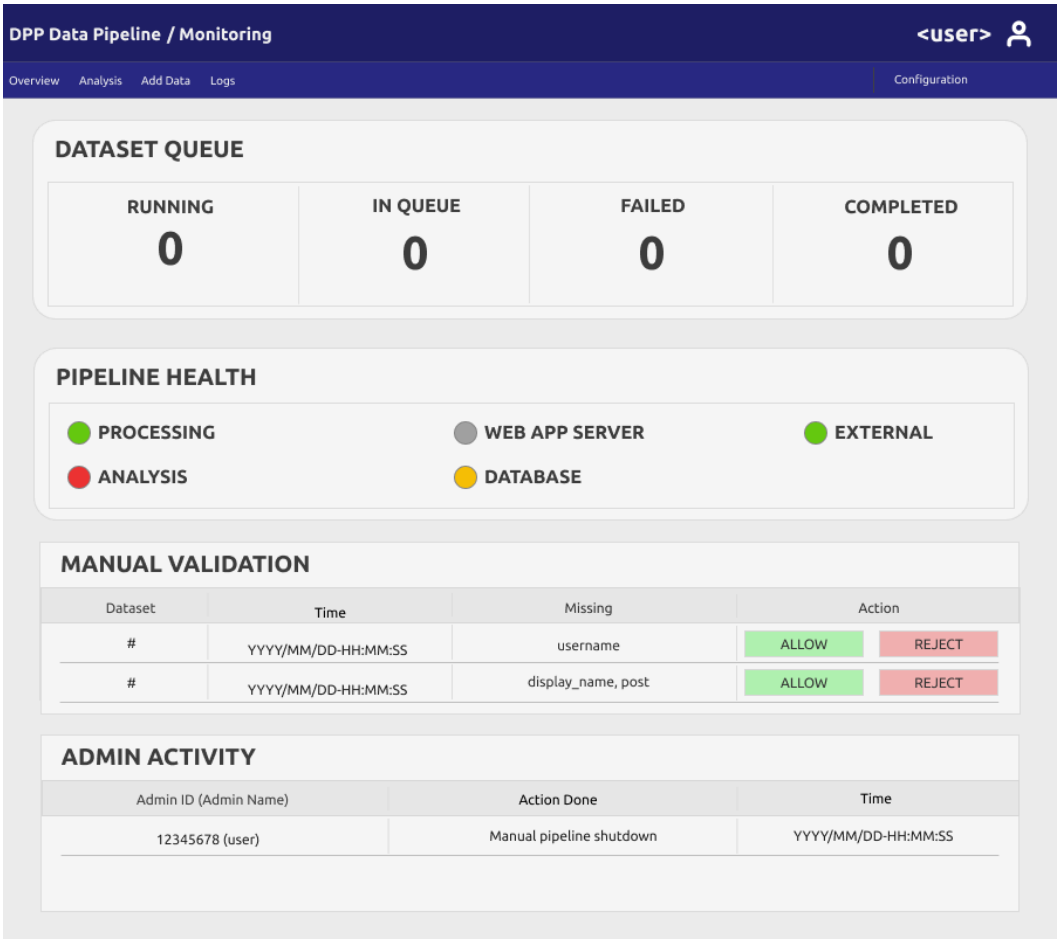


Figure F: System Monitoring Interface

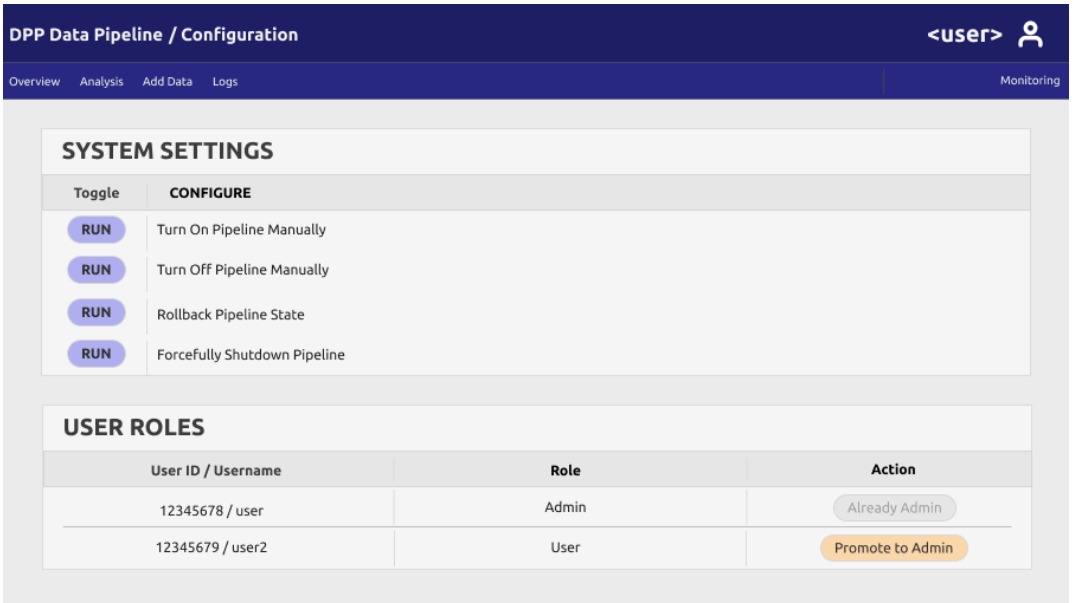


Figure G: System Configuration Interface

Going by the issues proposed in Section 11.4.3 of Pressman's *Software Engineering: A Practitioner's Approach*, the design addresses these issues through the following:

- **Response Time**: The interface itself is relatively fast due to the decoupling between each individual interface, meaning traversing between each interface should be quick. However, the response time for the software itself is expected to be relatively slow due to the nature of the pipeline, which takes in a lot of datasets and processes them in sequence. In practice, this is within expectations. The team has to guarantee that this "slowness" is consistent rather than fluctuating for the same amount of datasets.

- **Help Facilities**: This will be expanded upon in the future as we get more user feedback. In practice, the design can be expanded upon to clearly indicate what each color means, what each state of the pipeline is, and what each section tells. This can be done either through a dedicated interface serving as a user manual or including at the top (or bottom) of each section what they are responsible for in each interface.

- **Error Handling**: An error in the interface, or even pipeline itself, will be logged into the Monitoring interface of the system administrator, which enables them to configure the system through different ways: either restart the system, go back to a previous state, or if the error occurs due to a missing field, manual validation by the administrator. For normal users, they will be able to see if the pipeline's status is in error if its status in either the "Overview" or "Logs" is "Error". In which case, they can ask the system administrator for help in diagnosing the error.

- **Menu and Command Labeling**: The menu and commands are labeled clearly to specify what they do. For example, "Run Pipeline" in the "Add Datasets" interface will run the pipeline on the given datasets passed onto it (provided there is no queue). In addition, some commands are colored to immediately evoke what the user can do. A simple example is the green "Allow" and red "Reject" buttons for manually validating datasets in the Monitoring interface of the administrator view.

- **Application Accessibility**: The interface is designed to be accessible to the context the client works in. However, some precautions were taken to make it "more accessible". This included larger text sizes, color-coded statuses and buttons, and a sectioned design allowing people to view what they need to easily.

- **Internationalization**: The design's primary language is English, which should help with other people understanding what happens with the site aside from just with diagrams and charts. However, since the project itself is entirely for the benefit of the client firstly, internationalization is not a concern as of now.

Going with the **Aesthetic Design** of the web app, the only issue the interface deals with, going by Pressman's guidelines, is with respect to the scrolling bar. It was a decision about balancing navigation and layout, of which the former was chosen. However, all the other guidelines were followed:

- There is a reasonable amount of white space in between the page sections.
- Rather than placing the navigation bar on the side, as stated earlier, it was placed at the top so that the content was the emphasis.
- The content flows from top-left to bottom-right.
- There is a clear geography in the pages. Navigation links are grouped together in the navigation bar and similar content sections are grouped together in their own sections.

With regards to the design itself, readability was favored over decoration. This meant ensuring the contrast of the text from the background enabled legibility. Furthermore, the color of the header itself is part of the aesthetic design, as it indicates the privilege level the current user has.

To explain the **Navigation Design,** we can look at how "friendly" the design is with navigation, i.e., if the user requires less effort to move compared to doing anything else, then we can say it's quite user-friendly for navigation. Included below is a table of some typical usage scenarios and how many clicks are required to accomplish their specified task:

Possible Usage Scenarios		
Navigation Presses	Clerical Data Entry	Usage Scenario
2	2	A user inputs data into the pipeline and waits for the analysis to complete.
0	0	A user logs in to view the Overview.
3	2	A user inputs data into the pipeline, sees an error, and looks at the logs.
2	4	A user enqueues another dataset immediately after enqueueing the first one, and waits for the analysis to complete.
1	2	A user goes into the Analysis section, and uses the filter mechanism of Network Analysis.

1	1	The administrator does manual data validation for a single invalid dataset.
1	1	The administrator goes into the system configuration and turns off the pipeline manually.
1	1	The system administrator goes into the system configuration and elevates a user to administrator.

Table 3: Usage Scenarios a user or administrator might do.

In these possible usage scenarios, we have 11 navigation presses and 13 clerical data entries (button click, add dataset, etc.) done. This means below 50% (45.8333%, to be exact) goes to navigation, with the rest going into data entries.

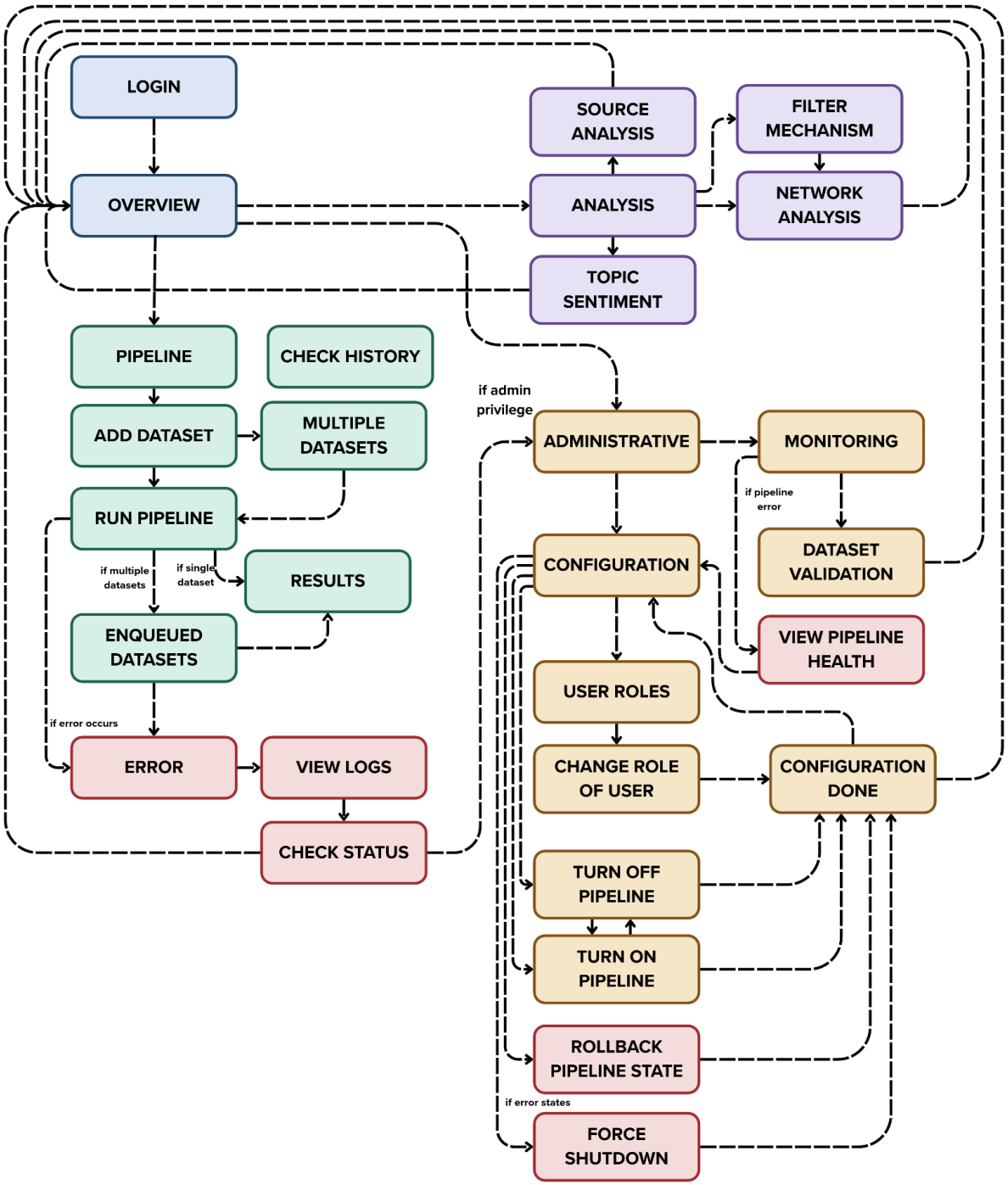


Figure H: Combined Navigation Semantic Unit

REFERENCES

Bunquin, J.B., Gaw, F., Baldo-Cubelo, J.T., Paragas, F., & San Pascual, M.R. (2022). *Digital Public Pulse: 2022 Philippine General Elections.* Philippine Media Monitoring Laboratory, College of Mass Communication, University of the Philippines.

Pressman, R. S. (2010). *Software Engineering A Practitioner's Approach*. McGraw-Hill Higher Education.